hashlock.

Security Audit

Balloon 2nd (DeFi)

September 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

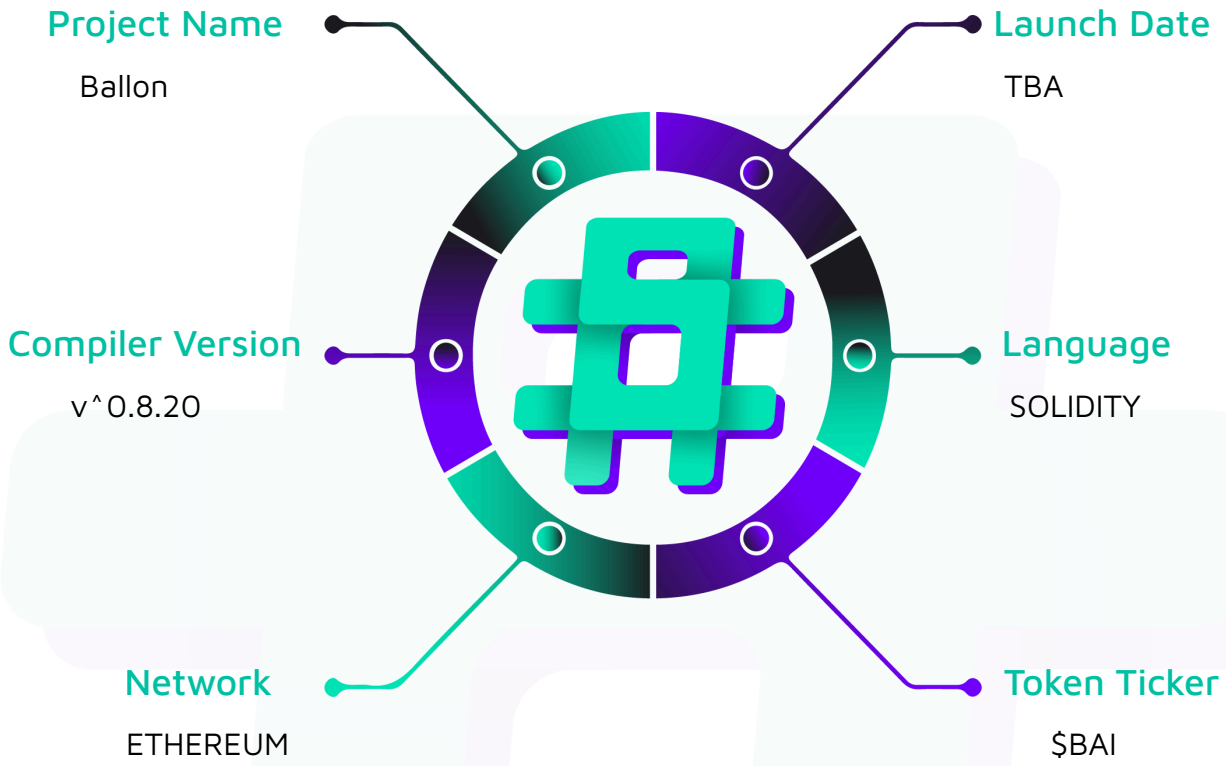
The Balloon team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code to ensure that the project's team and community that the deployed contracts are secure.

Project Context

Balloon Tech is a platform that develops DeFAI Agents, autonomous artificial intelligence systems that operate nonstop across multiple blockchain networks to optimize yield farming opportunities. These agents are designed to analyze real-time data, identify the best rates, and automatically allocate assets into curated vaults such as ETH-USDC, Gold, BlackRock, and BTC. By combining AI-driven strategies with human oversight, BalloonTech aims to maximize returns while reducing inefficiencies in decentralized finance. For example, their ETH-USDC vault currently offers competitive APYs around 18.2%, showcasing how automation and intelligence can create sustainable advantages in DeFi.

Project Name: Balloon
Project Type: Defi
Compiler Version: ^0.8.20
Website: https://www.balloontech.ai/
Logo:

Visualised Context:



Project Visuals:

Audit Scope

We at Hashlock audited the solidity code within the Ballon project. The scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Balloon Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**September, 2025**
Contract 1	DepositVault.sol
Audited GitHub Commit Hash	18f2b0737bb178b965d3864873c58791d70e965e
Fix Review GitHub Commit Hash	f84b58e6b54eaef31347440cb5c153d59358efcc

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section, and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

2 High severity vulnerabilities

1 Medium severity vulnerabilities

2 Low severity vulnerabilities

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
DepositVault.sol - ERC-4626 tokenized vault that time-locks all user deposits. - Owner controls deposit limits, a whitelist/denylist, and contract upgrades. - The owner can sweep all underlying assets to a designated multisig wallet at any time.	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Ballon project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Ballon project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] DepositVault#_canWithdraw - Withdrawal Lock Can Be Bypassed by Transferring Shares

Description

The contract's `time-lock` mechanism is incorrectly tied to the depositor's address instead of the shares themselves. This allows a user to deposit assets, receive locked shares, and then transfer those shares to a different address. The recipient, who has no lock record, can then immediately withdraw the underlying assets, completely bypassing the intended lock duration.

Vulnerability Details

The issue originates in the `_canWithdraw` function, which determines if an address is permitted to withdraw assets. The function incorrectly assumes that any address with a lock timestamp of zero should be allowed to withdraw.

```
function _canWithdraw(address user) internal view returns (bool) {

    // ...

    if (userLockTimestamp[user] == 0) {

        return true; // <-- THE FLAW

    }

    // ...

}
```

When a user deposits, their address is assigned a lock `timestamp`. However, the vault's shares are standard `ERC-20` tokens and can be freely transferred using the `transfer()` function. When locked shares are sent to a new address, the recipient's lock timestamp remains at the default value of `0`. As a result, the check `userLockTimestamp[user] == 0`

returns true for the recipient, granting them immediate withdrawal access to assets that should have remained locked.

Proof of Concept

```
function testLockBypassByTransferringShares() public {

        uint256 depositAmount = 100e18;

        // 1. User1 deposits and gets their shares locked.

        vm.startPrank(user1);

        asset.approve(address(vault), depositAmount);

        uint256 shares = vault.deposit(depositAmount, user1);

        vm.stopPrank();

        // 2. Verify User1 is indeed locked and cannot withdraw.

        assertFalse(vault.canUserWithdraw(user1));

        vm.startPrank(user1);

        vm.expectRevert(

                    abi.encodeWithSelector(DepositVault.WithdrawalLocked.selector,  user1,
block.timestamp + LOCK_DURATION)

        );

        vault.withdraw(depositAmount, user1, user1);

        vm.stopPrank();

        // 3. User1 transfers their "locked" shares to User2.

        // User2 has never deposited, so their userLockTimestamp is 0.

        vm.startPrank(user1);

        vault.transfer(user2, shares);

        vm.stopPrank();

        // 4. Verify shares are transferred.

        assertEq(vault.balanceOf(user1), 0);
```

```
        assertEq(vault.balanceOf(user2), shares);

        // 5. User2, who now holds the shares, can immediately withdraw

        // because their lock timestamp is 0, bypassing the lock.

        assertTrue(vault.canUserWithdraw(user2));

        uint256 user2AssetBalanceBefore = asset.balanceOf(user2);

        vm.startPrank(user2);

        vault.redeem(shares, user2, user2);

        vm.stopPrank();

        // 6. Verify User2 successfully withdrew the assets.

        assertEq(asset.balanceOf(user2), user2AssetBalanceBefore + depositAmount);

        assertEq(vault.balanceOf(user2), 0);

    }
```

Impact

The impact of this issue is critical as it completely negates the core functionality of the smart contract. The `time-lock` feature, which is the primary purpose of the vault, can be trivially bypassed.

Recommendation

To fix this issue, prevent the transfer of shares while they are still under a time-lock. This can be achieved by overriding the `_beforeTokenTransfer` hook provided by the `OpenZeppelin ERC20Upgradeable` contract. By adding logic to this hook, you can ensure that a transfer is only permitted if the sender's lock has expired..

Status

Resolved

[H-02] DepositVault#deposit - Deposit Lock Timer Resets on New Deposits, Enabling a Griefing Attack

Description

The contract unconditionally resets a user's lock timer every time a new deposit is made to their account. This can be triggered either by the user themselves making an additional deposit or, more critically, by a malicious user. An attacker can send a "`dust`" deposit to any victim's address, forcibly resetting their withdrawal lock and trapping their funds for an extended period.

Vulnerability Details

The issue exists within the `deposit()` and `mint()` functions. In both functions, the lock timer is reset for the receiver on every successful call, without checking if a lock is already active

```
function deposit(uint256 assets, address receiver) public virtual override returns (uint256 shares) {

    // ... deposit logic ...

    shares = super.deposit(assets, receiver);

    userLockTimestamp[receiver] = block.timestamp; // <-- THE FLAW

    // ...

}
```

Because the receiver is a user-supplied parameter, an attacker can call deposit(`1`, `victimAddress`). This action credits the victim's account with a single wei of a new share but, more importantly, `overwrites` their existing `userLockTimestamp`. This resets the lock on the victim's entire balance, not just the dust amount, effectively restarting the full lock duration against their will.

Proof of Concept

```
function testGriefingAttackResetsLockTimer() public {

        uint256 victimDepositAmount = 100e18;
```

```
        uint256 attackerGriefAmount = 1; // A single wei is enough to grief

        // 1. Victim (user1) deposits a large amount and their lock timer starts.

        vm.startPrank(user1);

        asset.approve(address(vault), victimDepositAmount);

        vault.deposit(victimDepositAmount, user1);

        uint256 initialUnlockTime = vault.getUserUnlockTime(user1);

        vm.stopPrank();

        // 2. Advance time to just before the victim's lock expires.

        uint256 timeJustBeforeUnlock = initialUnlockTime - 1 days;

        vm.warp(timeJustBeforeUnlock);

        // At this point, the victim can't withdraw yet, but is close.

        assertFalse(vault.canUserWithdraw(user1));

        // 3. Attacker (user2) deposits a tiny amount *to* the victim's account.

        vm.startPrank(user2);

        asset.approve(address(vault), attackerGriefAmount);

        // The key is that `receiver` is user1 (the victim)

        vault.deposit(attackerGriefAmount, user1);

        uint256 griefDepositTimestamp = block.timestamp;

        vm.stopPrank();

        // 4. VERIFY THE ATTACK: The victim's lock timestamp has been reset.

          assertEq(vault.userLockTimestamp(user1), griefDepositTimestamp, "Victim's lock
timestamp was reset");

        // The victim's new unlock time is far in the future.

        uint256 newUnlockTime = griefDepositTimestamp + LOCK_DURATION;

        assertEq(vault.getUserUnlockTime(user1), newUnlockTime, "Victim's unlock time was
extended");
```

```
          // 5. As a result, even after the original unlock time passes, the victim is
STILL locked.

        vm.warp(initialUnlockTime + 1);

         assertFalse(vault.canUserWithdraw(user1), "Victim is still locked after original
unlock time");

        // The victim must now wait for the *new* lock duration to expire.

        vm.startPrank(user1);

           vm.expectRevert(abi.encodeWithSelector(DepositVault.WithdrawalLocked.selector,
user1, newUnlockTime));

        vault.withdraw(victimDepositAmount, user1, user1);

        vm.stopPrank();
```

Impact

This issue allows for a highly effective and cheap griefing attack. A malicious actor can indefinitely prevent any user from withdrawing their funds by periodically sending them dust deposits.

Recommendation

We recommend implementing `per-deposit` locks so only new shares are locked.

Status

Resolved

Medium

[M-01] DepositVault#setLockDuration - Owner Can Unilaterally Change Lock Duration for All Depositors

Description

The contract owner has the ability to change the `lockDuration` at any time. This change immediately affects all users, including those who deposited funds under a previously defined lock period, altering the terms of their deposit.

Vulnerability Details

The issue lies in the `setLockDuration` function, which is protected by an `onlyOwner` modifier. This function allows the owner to set a new value for the `lockDuration` state variable without any restrictions

```
function setLockDuration(uint256 newLockDuration) external onlyOwner {

    uint256 oldDuration = lockDuration;

    lockDuration = newLockDuration;

    emit LockDurationUpdated(oldDuration, newLockDuration);

}
```

Because the withdrawal eligibility check in `_canWithdraw` directly uses this global `lockDuration` variable, any change made by the owner will apply to all existing locked positions.

Impact

This function creates a risk, particularly in the event that the owner's account is compromised. A malicious actor can alter the lock duration to an extremely large value. This would effectively trap all user funds in the vault indefinitely.

Recommendation

If the flexibility to change the lock duration is a business requirement, the risk should be mitigated by introducing configurable upper and lower bounds. This approach prevents an attacker with a compromised key from setting an extreme value while still allowing for reasonable adjustments.

Status

Resolved

Low

[L-01] DepositVault#initialize - Missing Input Validation in the initialize Function

Vulnerability Details

The initialize function lacks necessary checks on its input parameters. It is possible to initialize the contract with a `multisigWallet_` of `address(0)` and a `lockDuration_` of `0`. This can lead to failed transactions when trying to use the multisig wallet and can silently disable the contract's core `time-locking` feature from the moment of deployment.

Recommendation

Add input validation to the initialize function to ensure critical parameters are set to sensible values. Specifically, require that the `multisigWallet_` is not the zero address and that the `lockDuration_` is greater than zero.

Status

Resolved

[L-02] DepositVault#transferAssets - Use SafeERC20 for admin asset transfers

Vulnerability Details

Admin sweep functions call `ERC20` transfer directly, which can silently fail on `non-standard` tokens; while the asset is expected to be standard and trusted, using `SafeERC20` is safer and consistent with the `ERC4626` internals.

Recommendation

Replace raw transfer calls in `transferAssets` and `transferAllAssets` with `SafeERC20Upgradeable.safeTransfer` for the vault asset.

Status

Resolved

QA

[Q-01] component#function - Redundant NotAuthorized Error

Vulnerability Details

The contract defines a custom error named `NotAuthorized`, but it is never used within the codebase. The imported `OwnableUpgradeable` contract already handles access control for owner-only functions, reverting with its own specific error, which makes the custom NotAuthorized error redundant.

Recommendation

Remove the `NotAuthorized` error definition to improve code clarity and reduce the contract's deployment size..

Status

Resolved

Centralisation

The Ballon project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Balloon project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au